

OnlyDiffs architecture

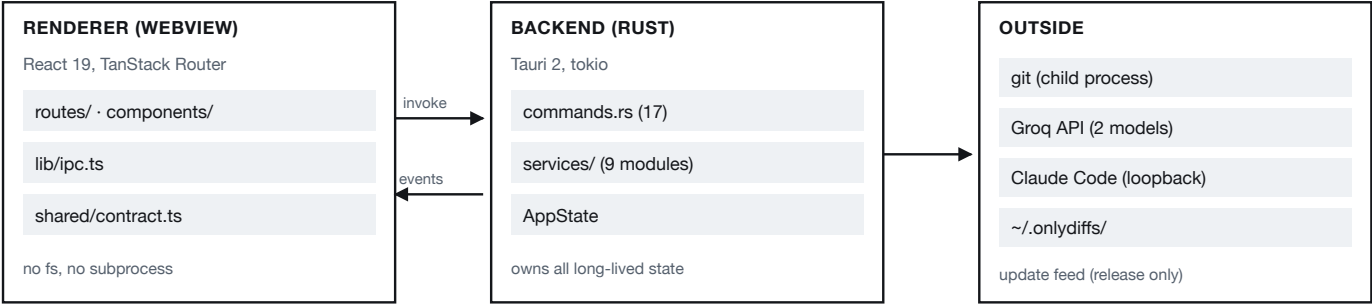
A desktop git diff viewer built on Tauri 2, with a Rust backend that shells out to `git` and a React renderer that never touches the filesystem. This is the map: what the pieces are, which one calls which, and where the decisions live.

Version 0.1.6 · 2,926 lines of Rust, 5,163 of TypeScript in the desktop app · 52 Rust tests, 21 renderer tests

1. The shape of the thing

OnlyDiffs shows you the working tree of one git repository and lets you talk to a Claude Code session about a specific line of it. That is the whole product. Everything below serves those two jobs.

The app runs as two processes that Tauri glues together. A Rust backend owns all state that outlives a render, spawns every `git` child process, and holds the only network clients. A React renderer draws, and asks the backend for data through a typed command bridge. The renderer has no filesystem access and no way to run a subprocess, which is the point: a compromised page cannot reach the repository except through the seventeen commands the backend chooses to expose.



FOUR BACKGROUND LOOPS

1. filesystem watcher, 300ms debounce, pushes repo:changed
2. project icon resolver, once per repo, pushes project:icon-changed
3. Claude channel poll, every 4s, renderer-driven
4. update check, once at +5s, release builds only

Processes and the traffic between them. Arrows left of centre are the command bridge; everything right of centre is the backend talking to the world.

2. Repository layout

A bun workspace driven by Turbo. Four packages, only one of which is the actual app.

PATH	WHAT IT IS
apps/desktop	The product. React renderer in <code>src/</code> , Rust backend in <code>src-tauri/</code> .
apps/web	Next.js marketing site. Shares nothing with the desktop app but the design system.
packages/ui	Design system on react-aria-components and tailwind-variants. Buttons, inputs, dialogs, loaders.
tools/oxlint	Anti-slop lint plugins, wired into the root <code>oxlint</code> task.
docs/research	Research notes. One file so far, on why GitHub cannot give a repository its own avatar.

Turbo passes exactly three environment variables through to tasks: `ONLYDIFFS_REPO_PATH`, `ONLYDIFFS_STATE_DIR`, and `GROQ_API_KEY`. That list is worth knowing, because those three are the only knobs the app reads from its environment.

3. The command bridge

This is the most opinionated part of the codebase and the part most worth copying.

Tauri commands normally return `Result<T, E>`, and a returned `Err` gets stringified on its way across. That flattens a structured failure into prose and throws away the discriminant. OnlyDiffs refuses to lose that. Every command returns `Result<IpcResult<T>, ()>`, where the error case is a value inside a successful response:

```
{ ok: true, value: T }
{ ok: false, error: { _tag: "GitError", message: "..." } }
```

The renderer's `call()` in `lib/ipc.ts` unwraps that and throws an `IpcError` carrying both `tag` and `operation`. Callers that only want a string read it as an ordinary `Error`. Callers that need to branch match on the tag. A genuine Tauri rejection, meaning a missing command or a payload that failed to deserialize, folds into the same error type with the tag `ChannelError`.

`AppError` has nine variants and `tag()` maps each to a string. Those strings are load-bearing contract, not debug output.

VARIANT	TAG	RAISED WHEN
Git	GitError	A git invocation failed or could not be spawned.

VARIANT	TAG	RAISED WHEN
WorkTree	WorkTreeError	Reading or writing the working tree failed.
InvalidPath	InvalidPathError	A renderer-supplied path escaped the repository.
CommitMessage	CommitMessageError	Groq generation failed, or the diff was too large.
ClaudeChannel	ClaudeChannelError	No live session, or the loopback send failed.
Clipboard	ClipboardError	The clipboard plugin refused a write.
NoProjectOpen	NoProjectOpenError	Nothing is open. The renderer shows the picker.
InvalidProject	InvalidProjectError	The path is not a git repository.
Updater	UpdaterError	A check, download, or install went wrong.

The seventeen commands

`shared/contract.ts` holds a `Command` map from camelCase key to the Rust function name, and `CommandName` is derived from it. Nothing else is reachable from the page.

COMMAND	DELEGATES TO	RETURNS
<code>get_diff</code>	<code>diff::get_diff</code>	<code>RepoDiff</code>
<code>get_file_contents</code>	<code>diff::get_file_contents</code>	<code>FullFileContents</code>
<code>get_history</code>	<code>history::get_history</code>	<code>Vec<Commit></code>
<code>stage_file</code>	<code>diff::stage_file</code>	<code>()</code>
<code>commit_all</code>	<code>diff::commit_all</code>	<code>String</code>
<code>list_files</code>	<code>file_tree::list_files</code>	<code>Vec<String></code>
<code>generate_commit_message</code>	<code>commit_message::generate</code>	<code>String</code>
<code>send_claude_message</code>	<code>claude_channel::send</code>	<code>String</code>
<code>claude_status</code>	<code>claude_channel::status</code>	<code>ClaudeChannelStatus</code>
<code>list_projects</code>	<code>workspace.list</code>	<code>Vec<Project></code>
<code>current_project</code>	<code>workspace.current_project</code>	<code>Option<Project></code>
<code>open_project</code>	<code>workspace.open</code>	<code>Project</code>
<code>forget_project</code>	<code>workspace.forget</code>	<code>()</code>
<code>set_theme</code>	<code>window.set_theme</code>	<code>()</code>
<code>write_clipboard_text</code>	clipboard plugin	<code>()</code>
<code>check_for_update</code>	<code>updater::check</code>	<code>UpdateStatus</code>
<code>install_update</code>	<code>updater::install</code>	<code>()</code>

Why two of these look out of place. `write_clipboard_text` exists because `navigator.clipboard` needs a secure context, and the packaged app loads from a custom protocol where it does not have one. `set_theme` exists because the OS draws the window frame, so an in-app theme that disagrees with the system one leaves the title bar out of step.

Payloads deserialize into request structs before any service sees them: `GetFileContentsRequest`, `StageFileRequest`, `GetHistoryRequest`, `SendClaudeMessageRequest`, `CommitAllRequest`, `OpenProjectRequest`, `ForgetProjectRequest`, `SetThemeRequest`. A tampered renderer cannot hand an arbitrary shape to `git`.

4. Backend

AppState and startup

`lib.rs` builds one `AppState` and Tauri manages it for the process lifetime.

FIELD	PURPOSE
<code>workspace</code>	Which repository is open, and the remembered list of them.
<code>watcher</code>	The active filesystem watch. Moves when the project changes.
<code>http</code>	One pooled <code>request::Client</code> shared by Groq and the Claude channel.
<code>icon_resolution</code>	A mutex that stops startup and a fresh <code>open_project</code> from running overlapping icon passes.
<code>pending_update</code>	The update handle from the last check, held so installing applies the version the user was offered.

Startup runs in this order:

- `Workspace::from_env()` reads `ONLYDIFFS_STATE_DIR` and `ONLYDIFFS_REPO_PATH`, loads `projects.json`, and opens either the pinned repository or the most recently used one that still exists on disk.
- `setup()` starts watching that repository. This happens here rather than in `AppState::new` because there is no `AppHandle` to emit through yet.
- The window is built at 1100x760, minimum 720x480, and **hidden**.
- A task sleeps 3 seconds and shows the window regardless. That is a safety net for a renderer that throws before it can show itself.

5. A second task sleeps 500ms, then starts icon resolution, so Groq calls never delay the first frame.
6. The renderer shows the window itself, on the second animation frame after first render. That is the earliest moment the page is genuinely painted.

`on_navigation` runs `is_internal()` on every attempted navigation. `tauri:`, `asset:`, and loopback HTTP pass. Everything else is handed to the system browser and the navigation is refused, so no link can replace the app with a web page.

Services

MODULE	LINES	RESPONSIBILITY
git.rs	65	The single place that spawns <code>git</code> .
diff.rs	419	Status walk, per-file patches, file contents, staging, committing.
workspace.rs	355	Which repository is open, and the persisted project list.
project_icon.rs	687	Finds repository artwork and asks a vision model to pick one.
claude_channel.rs	247	The loopback bridge to a Claude Code session.
watcher.rs	193	Debounced, gitignore-filtered filesystem watch.
commit_message.rs	142	Generates a commit message from the working tree via Groq.
updater.rs	79	Finds and installs a newer release.
history.rs	63	Commit log parsing.
file_tree.rs	24	Every path in the repository, flat.

git.rs

Three functions: `run_in(cwd, args)`, `run(workspace, args)` which resolves the current repository first, and `show_file(workspace, revision, path)` where an empty revision means the index.

Two details here are not stylistic. It uses `output()` rather than spawning and waiting, because `output()` drains stdout and stderr while it waits, and a patch larger than the OS pipe buffer would otherwise deadlock the child forever. And it treats non-empty stdout as success even on a non-zero exit, because `git diff --no-index` exits non-zero precisely when it did produce a patch. Decoding is deliberately lossy, so one stray byte renders a file rather than blanking the card.

diff.rs

The busiest module. Five public functions.

`get_diff` is the startup read. It runs three git commands concurrently through `try_join! : rev-parse --abbrev-ref HEAD` for the branch, `log -1` for the head line, and `status --porcelain -z -uall` for the changes. The `-uall` matters, because without it an untracked directory collapses into a single record and you cannot diff a directory.

`parse_porcelain` splits that status output into `Side` records, one per changed half. A path that was edited, staged, then edited again produces two of them, because those are genuinely two different patches. Then the per-file patches run through `stream::buffered` at a concurrency of 8, and results sort by path with the staged half first.

`get_file_contents` is deliberately a separate command. If it were folded into `get_diff`, startup would read every changed file in full. Instead the cards ask for their own contents as they approach the viewport.

Path safety lives in `is_safe_repo_path` and `validate_path`. Paths arrive from the renderer, so `..`, a leading separator, and a Windows drive letter are all rejected before anything is handed to `git show` or joined onto the root.

`commit_all` runs `git add -A` and then `git commit -m`, rather than `commit -a`, because `-a` skips untracked files and that is not what a user means by "commit all". The message goes through `-m` as one argument, so nothing in it reaches a shell or is read as a flag.

`commit_message_diff` assembles the whole working tree into one annotated document for the model, with each patch under a `### staged: , ### unstaged: , or ### untracked:` heading.

workspace.rs

Holds `current: Mutex<Option<PathBuf>>` and `recents: Mutex<Vec<StoredProject>>`, persisted to `~/.onlydiffs/projects.json` at schema version 1. Opening a path walks up to find the repository root, so opening any file inside a checkout opens the checkout.

Two ordering decisions are easy to get wrong. The project list keeps first-opened order rather than most-recent order, so switching projects never makes the rail jump around under the cursor. Restoring at startup uses `last_opened_at` separately, and that timestamp is kept strictly increasing even when two opens land in the same millisecond. Entries whose folder has since vanished are skipped rather than deleted, so unplugging a drive does not erase history.

`project_icon_jobs()` and `record_icon_scan()` are the icon resolver's view of this store. Jobs sort the currently open project first.

watcher.rs

Before this existed the diff only refreshed when a route loader ran, which made a stale view the default: an agent could rewrite twenty files and the screen would not move until you navigated. Now the backend pushes.

`ChangeFilter` applies the repository's own ignore rules through the `ignore` crate, so `node_modules` and `target` never produce an event, and `is_git_signal` lets through the few `.git` entries that actually mean something while dropping the rest. What survives is debounced at 300ms and emitted once as `repo:changed`. A thirty-file rewrite is one refresh.

claude_channel.rs

Claude Code sessions advertise themselves by writing a JSON file into `~/onlydiffs/claude-channels/`. A registration carries a schema version, a port, a bearer token, a pid, and the working directory.

`registrations()` reads that directory and keeps a file only if the schema version matches, the port is non-zero, the token is non-empty, the `cwd` normalizes to the open repository, and the process is still alive. The liveness check is last on purpose, because it is the only one that costs a syscall. Survivors sort newest first. Half-written and stale files are skipped in silence rather than raised, since the next candidate may well work.

`send()` posts to `127.0.0.1` with the token, capped at 64KB and 10 seconds. The channel is one-way by design. It resolves with a message id, which is only useful for correlating logs, and whatever Claude does next happens in Claude Code.

The two Groq integrations

Both read `GROQ_API_KEY` from the process environment. The key never reaches the renderer bundle.

	COMMIT MESSAGE	PROJECT ICON
Model	openai/gpt-oss-120b	qwen/qwen3.6-27b
Trigger	User, from the palette	Background, once per repository
Input cap	240,000 chars of diff	3 images, 256px each
Reasoning	low	none
Timeout	120s	45s
Failure	Surfaced to the user	Logged, cube fallback kept

The icon resolver is the more interesting of the two. It ranks candidate image files by name (`icon` and `logo` score high, anything matching `screenshot` or `banner` is penalized 100), takes the top 32, renders each to a 256px PNG, drops byte-identical duplicates, and sends at most three to the vision model. Three is not a taste decision. Groq rejects a request carrying more than three images. SVGs rasterize through `resvg` with no resource directory, so a candidate cannot read sibling files or fetch a remote URL while being rendered.

The model may also decline every candidate, and that is a real answer rather than a failure. A repository whose only image is a screenshot keeps the cube. The result is keyed by a hash of the candidate shortlist, so an unchanged repository is never sent twice. A failed Groq call stops the whole pass rather than retrying per repository, because a rejected key or a retired model will fail identically for every project still queued.

5. Renderer

Entry and routing

`main.tsx` awaits `currentProject()` at module scope and picks the starting route from the answer: `/` when nothing is open, `/diff` otherwise. Deciding it here rather than redirecting out of `/` keeps the picker reachable, since the sidebar's link to `/` still lands there.

The router uses `createMemoryHistory`, not hash history. WebKit binds Backspace to its own page history even while a text input is focused, so hash routes meant that deleting text in the command palette could navigate out of the repository. A desktop window has no URL to restore anyway.

StrictMode is on, and the comment explaining why the first diff blanks in dev is worth reading before anyone "fixes" it: `@pierre/diffs` writes imperatively into a shadow root, and the simulated unmount tears that down.

ROUTE	COMPONENT	LOADER
__root	Outlet only	none
/	Welcome, the project picker	<code>listProjects</code>
/_app	Layout: rails, nav, toolbar	<code>getDiff</code> , <code>listFiles</code> , <code>listProjects</code> in parallel
/_app/diff	AppNoSelection, the changed-file list	reads the layout's data
/_app/file/\$	SingleFile, one path's diff	reads the layout's data

`/_app` is a pathless layout route, so it contributes no URL segment. Both content routes render inside it and share its loader data, which is why navigating between files does not refetch the diff. Its loader redirects to `/` when `currentProject()` returns null. Everything below it refreshes together, because every refresh path calls `router.invalidate()` on the same loader.

Components

FILE	LINES	WHAT IT DOES
app-command-menu	413	Cmd+K palette: stage, commit, generate message, switch project, theme, install update.
file-diff-card	249	One file's diff, lazily loaded and error-boundaried.
app-file-tree	212	Virtualized tree with a filter box.
app-toolbar	206	Footer status, and the line composer that talks to Claude.
project-sidebar	159	The 32px project rail down the left edge.
app-sidebar-nav	103	Breadcrumb path, deleted badge, R and S hotkeys.
theme-provider	95	Theme state, and the call that repaints the native frame.

FILE	LINES	WHAT IT DOES
app-sidebar	87	Header, totals, hosts the file tree.
app-no-selection	48	The changed-file list shown before you pick one.
project-tile	37	Squirrel-clipped tiles for the rail, via Lisse.

file-diff-card.tsx

Three mechanisms worth knowing. An `IntersectionObserver` with a 600px margin sets `shouldRender`, so a repository with eighty changed files does not fetch eighty file bodies at once. A module-level `WeakMap` keyed on the `FileChange` deduplicates in-flight content requests, so two cards for the same object share one promise and the entry is collected with the object. And `DiffBoundary` is a class error boundary, because `@pierre/diffs` parses patches at render time and a malformed one should cost a card, not the window. The single-file route skips the observer entirely. There is exactly one card and it is always in view, and waiting on an observer there can strand the card if the pane has not been laid out when the observer starts.

app-file-tree.tsx

`lib/file-tree.ts` splits the work in two on purpose. `buildFileTree` is $O(\text{paths})$ and runs once per repository load. `flattenTree` runs on every expand, collapse, and keystroke, so it stays cheap. Runs of single-child directories fold into one row, which is why you see `.github/workflows` rather than two levels.

Rows are 26px. Virtualization switches on above 150 rows rather than always, since a virtualizer costs more than it saves on a short list. The filter goes through `useDeferredValue` so typing never blocks on a re-flatten.

Contexts and hooks

MODULE	SHARES	WHY A CONTEXT
line-reference	The last diff line clicked	The click is deep under the route outlet; the input is in the toolbar beside it.
diff-layout	Split or unified	Toggled by a hotkey in the nav, read by every card.
update	The release on offer	The footer mentions it, the palette installs it.
theme-provider	Theme, and the native frame call	Read by the palette, applied to the document root.

`useRepoWatch` listens for `repo:changed` and coalesces. A signal arriving mid-refresh sets a flag instead of starting a second pass, and the loop runs once more when the first settles, so any number of bursts collapse into at most one queued refresh and the last pass always reads the final state. It also refreshes on window focus, because a watch can be missed on a network mount or after a machine resumes, and coming back to the window is exactly when you would notice a stale view.

`useProjectIcons` is the same shape but simpler: listen for `project:icon-changed`, invalidate, done.

Keyboard

KEY	ACTION	DEFINED IN
Cmd+K	Toggle the command palette	app-command-menu
Cmd+B	Toggle the file sidebar	ui/sidebar
Cmd+Enter	Stage the current file	routes/_app.file.\$
R	Refresh	app-sidebar-nav
S	Split or unified	app-sidebar-nav

R and S have no on-screen control at all. They are the only way to reach refresh and the split toggle, which is a real discoverability gap rather than a deliberate minimalism.

6. State on disk

PATH	WRITTEN BY	HOLDS
<code>~/.onlydiffs/projects.json</code>	Backend, <code>workspace.rs</code>	Project list, open timestamps, resolved icons as data URLs, icon scan hashes.
<code>~/.onlydiffs/claude-channels/<pid>.json</code>	The MCP script, per session	Port, bearer token, pid, cwd, schema version, start time.

Both live under one directory the user can delete without losing anything but convenience. `ONLYDIFFS_STATE_DIR` redirects the first, which is what keeps the test suite out of a real home directory.

7. The Claude Code channel

`scripts/claude-channel.ts` is the other half of the bridge, registered as a user-scope MCP server by `bun run channel:setup`. One script does two jobs at once.

It connects to Claude Code over stdio as an MCP server, declaring an experimental `claude/channel` capability and instructions telling Claude that arriving messages come from the user working in this repository. Separately it starts a Bun HTTP server bound to `127.0.0.1` on port 0, so the OS picks a free port. A 32-byte random token guards it, compared with `timingSafeEqual`. Then it writes its registration file with mode bits restricted, and removes it on exit.

So the full path from a click to Claude runs: a diff line click sets the line reference, the toolbar composer sends the text, `send_claude_message` reaches `claude_channel::send`, which finds the live registration for the open repository and POSTs to its loopback port with the token, and the script forwards it into the MCP session. Nothing comes back.

8. Tests

SUITE	COUNT	COVERS
tests/workspace.rs	21	Open, restore, forget, ordering, store migration, icon persistence.
project_icon (unit)	12	Name ranking, path safety, SVG rasterizing, dedupe, model refusal.
tests/watcher.rs	10	Ignore rules, git internals, debouncing.
tests/diff.rs	6	Path escapes, renames, both halves of one path, history order.
tests/claude_channel.rs	3	Registration validation.
src/lib/*.test.ts	21	File tree building and flattening, file icon resolution.

The Rust integration tests drive the services directly rather than going through Tauri, which is why `contract`, `error`, and `services` are all `pub` from the library. `Workspace::new` takes its state directory and startup repository as arguments instead of reading the environment, so the suite runs in parallel without racing over process-wide variables. One test exists purely to assert the store never lands in a real home directory.

9. Build and release

`bun run dev` at the root runs every workspace through Turbo; `dev:desktop` filters to the app. Inside `apps/desktop`, `tauri dev` starts Vite on a fixed port 1420 and fails loudly on a clash, because silently serving somewhere the window will not look is worse than stopping.

`bun run build` is typecheck, then tests, then the Vite build. `bun run dist` bundles: dmg and app on macOS, nsis on Windows, appimage and deb on Linux. The updater plugin is compiled in only for desktop release builds, since it reads a signing key at startup that a dev build has no use for.

Three webviews, and it is worth remembering which is which. Windows gets WebView2, which is evergreen Chromium. macOS gets WKWebView and Linux gets WebKitGTK, both WebKit. Two of the three targets are WebKit, so a WebKit-only rendering bug is the default case here, not an edge case.

10. Things I would flag to a new contributor

The contract files are duplicated by hand. `contract.rs` and `contract.ts` mirror each other with a comment in each asking you to update the other, and nothing enforces it. A field added on one side and forgotten on the other typechecks fine and fails at runtime. That is the highest-risk seam in the codebase.

Failure handling is not uniform, and mostly for good reasons. A failed update check stays silent because the user did not ask and offline is a laptop's normal state. A failed icon resolution logs and keeps the fallback. A failed commit-message generation goes straight to the palette. Those are three different policies, and each is defensible, but you have to read each one to know which you are in.

The renderer holds no cache of its own. Every refresh path funnels into `router.invalidate()` on the `/_app` loader, which is why refresh, the watcher, focus, project switching, and icon resolution all take the same code path. It is a simple design that has held up, though it does mean any refresh re-reads the entire diff.

11. Architecture review: deepening opportunities

Four places where the current shape costs something. Each is judged on *depth*, meaning how much behaviour a caller or a test gets per unit of interface it has to learn, and on where the *seam* sits. The repository has no `CONTEXT.md` and no ADRs, so the domain names below are the ones the code already uses: Workspace, Project, RepoDiff, Side, ProjectIcon.

Scope came from the churn. `app-command-menu.tsx` is the most-edited source file in the last sixty commits, the IPC seam has been rewritten twice (Electron to Tauri, then Effect out of the IPC layer), and the project and icon cluster is where the newest 829 lines landed.

A. Opening a project is written three times, and the three disagree

STRONG

FILES

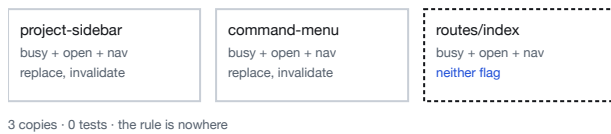
project-sidebar.tsx:30 , app-command-menu.tsx:128 , routes/index.tsx:65

PROBLEM

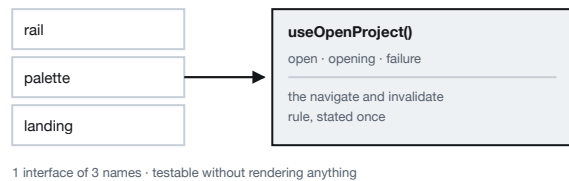
All three run the same sequence: mark busy, call `openProject`, navigate to `/diff`, refresh loaders, catch, clear busy. They do not agree on the last details. The rail and the palette pass `replace: true` and then call `router.invalidate()`; the landing page passes neither. Both behaviours are defensible, because entering `/_app` from `/` runs the layout loader anyway and there is no file route to replace. Nothing says so. A reader cannot tell the difference between a considered choice and an oversight, which is what makes it a locality problem rather than a bug. `OtherProjectsSection` is the sharper symptom. It takes seven props, four of them lifecycle callbacks (`onStart`, `onOpened`, `onError`, `onFinish`), to wrap about thirty lines. Its interface is nearly as large as its implementation, which is the definition of a shallow module. Deleting it would move those lines back to its parent rather than concentrate anything.

- SOLUTION** One project-opening module, `useOpenProject()`, returning `{ open, opening, failure }`. The navigation rule moves inside it, stated once: replace the route when already inside `/_app`, invalidate when the loader will not re-run on its own. The three call sites keep their own presentation and lose their copy of the sequence. `OtherProjectsSection` collapses into the palette that holds it.
- BENEFITS** Leverage across three call sites for one interface of three names. Locality for the ordering rule, which is currently implicit in three files. For tests this is the clearest win: today the sequence is only reachable by rendering three components, and it is untested in all three. Behind one module it becomes one thing to test.

BEFORE · THE SEQUENCE LIVES IN THREE PLACES



AFTER · ONE MODULE, THREE CALLERS



B. The icon resolver's decisions cannot be reached through its interface

STRONG

FILES `services/project_icon.rs:435 (resolve_job), :472 (resolve_missing)`

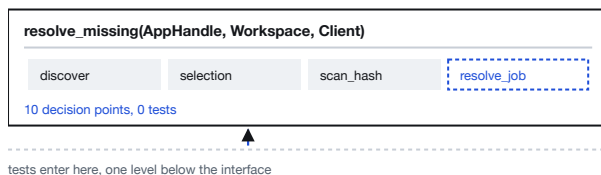
PROBLEM `resolve_missing(&AppHandle, &Workspace, &Client)` needs a live Tauri handle and a working Groq key to run at all, so no test calls it. The twelve tests in this module reach past the interface to internals instead: `discover`, `selection`, `scan_hash`, `name_score`, `request_content`. Neither orchestration function is imported by any test. That leaves the decisions about whether a project gets an icon at all completely uncovered: the unchanged-hash short circuit, recording an empty shortlist, recording a model refusal, and the split between a per-project failure that continues the loop and a Groq failure that stops it. Ten decision points, zero tests. This is also the newest code in the repository and the error handling was rewritten most recently, which is exactly the combination that should worry someone.

SOLUTION Put a seam where the model choice happens: a one-method `ChooseIcon` trait with the Groq call behind it, and have the resolver take a notifier callback rather than an `AppHandle`. The pipeline stages, discovery, choice, persistence and notification, stay where they are; only the two that reach outside the process move behind adapters.

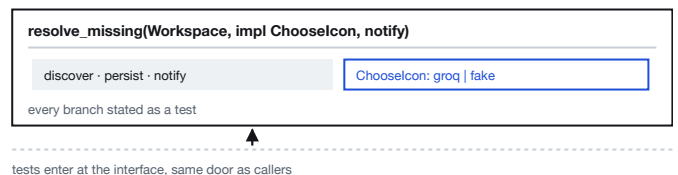
BENEFITS The interface becomes the test surface. Each branch turns into a sentence a test can state: a repository with no artwork records its hash and emits nothing, a refusal keeps the fallback, a rejected key stops the pass instead of firing one doomed request per project. Locality improves too, because the retry policy stops being spread between the resolver and the shape of the `Workspace` store.

Worth being honest about the cost. One adapter is a hypothetical seam; two make it real. Here the second adapter is a test double, which is the weaker justification for a seam. Ten untested decision points in the newest code is what tips it.

BEFORE · TESTS REACH PAST THE INTERFACE



AFTER · THE INTERFACE IS THE TEST SURFACE



C. The IPC seam has one interface spread across five files

STRONG

FILES `contract.rs, commands.rs, error.rs, shared/contract.ts, lib/ipc.ts`

PROBLEM Both contract files open with a comment calling themselves the mirror of the other. Only half of that is true. The response types do mirror `contract.rs`, but the four request shapes on the TypeScript side, `GetFileContentsRequest`, `GetHistoryRequest`, `StageFileRequest` and `IpFailure`, have no counterpart there at all. Their Rust definitions live in `commands.rs` and `error.rs`. Adding one command means editing five files, and nothing checks that any of them agree.

I compared the two files field by field while writing this. They are in sync today, so this is prevention rather than a live defect. The failure mode is quiet: a field added on one side and forgotten on the other typechecks on both sides and fails at runtime.

SOLUTION Derive the TypeScript from the Rust, with `ts-rs` or `tauri-specta`, so the interface has one definition and the second adapter is generated rather than maintained. If that is too much for now, the cheap version is a test that parses both files and asserts the field sets match, which is roughly twenty lines and catches the same class of drift.

BENEFITS Locality for the whole seam: one place to change a payload instead of five to remember. Leverage for callers, since the command names and their argument shapes stop being three separate facts to keep in your head. The test story is the clearest part, because drift moves from something a reviewer has to notice to something CI fails on.

D. One invariant, two implementations: repository-relative path safety

WORTH EXPLORING

FILES	<code>services/diff.rs</code> (<code>is_safe_repo_path</code>), <code>services/project_icon.rs</code> (<code>is_safe_relative</code>)
PROBLEM	Both enforce the same rule, that a path must stay inside the repository, and they do it differently. <code>is_safe_repo_path</code> works on a string and rejects an empty value, a leading separator, a Windows drive letter, and any <code>..</code> segment. <code>is_safe_relative</code> works on a <code>Path</code> and requires every component to be <code>Normal</code> . They disagree on real input: <code>./assets/icon.png</code> passes the first and fails the second. Neither is wrong, but the rule now lives in two places, and tightening it later means finding both.
SOLUTION	One path-safety module both call, with the two entry points kept because the callers genuinely differ: paths arrive from the renderer as strings and from <code>git ls-files</code> as paths.
BENEFITS	Locality for a security-relevant rule, which is the kind worth having in one place. The leverage is small, two callers, which is why this is worth exploring rather than urgent. Both call sites already have a test, so those tests move rather than multiply.

Top recommendation

Start with **A**. It is the smallest change, it sits in the most-edited file in the repository, and it is the only one of the four where three copies of the same operation are already behaving differently in shipped code. It also has the cleanest deletion test in both directions: removing `OtherProjectsSection` moves complexity rather than concentrating it, which marks it as shallow, while removing the proposed module would scatter the sequence back across three files, which marks that one as earning its keep.

Take **B** next, and take it seriously. **A** is about tidiness; **B** is about not knowing whether the newest code in the repository works. Ten decision points with no test, in error-handling logic that changed most recently, is the highest correctness risk here.

C is prevention and can wait for the next time someone adds a command, which is the moment its cost is felt. **D** is worth doing whenever `project_icon.rs` is next open.

Generated from the source at 0.1.6. Line counts and constants were read from the tree, not recalled. Where a number appears here it came from the file it describes.